САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа "ДЗ 4: Технический дизайн микросервисной архитектуры"

Выполнил:

Недиков Михаил

БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Задание:

- Разделите ваше монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service
- Спроектируйте микросервисную архитектуру: отразите взаимосвязь между микросервисами, опишите способы их взаимодействия друг с другом
- Спроектируйте разделение вашей БД на обособленные БД
- Спроектируйте и опишите эндпоинты, необходимые для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI
- Опишите варианты запросов и ответов, задокументируйте возможные ошибки
- В результате вы должны сформировать документ, в котором будут описаны конкретные требования и шаги для достижения разеделения вашего монолитного бэкенд-приложения на отдельные микросервисы

Ход работы

1. Разделение монолита на микросервисы

Чтобы устранить связанность и соблюсти принцип database-per-service, монолит сервиса аренды недвижимости разделён на четыре независимых микросервиса:

Сервис 1: Auth & User Service - отвечает за регистрацию, аутентификацию, верификацию JWT-токенов и управление профилем пользователя (регистрация, логин, получение и обновление данных).

Сервис 2: Property Service - управляет объектами недвижимости: создание, редактирование, удаление объявлений, получение списка с фильтрацией, а также управление локациями, атрибутами и изображениями.

Сервис 3: Booking Service — отвечает за заявки и бронирования, проверку доступности объектов недвижимости и хранение статусов бронирований.

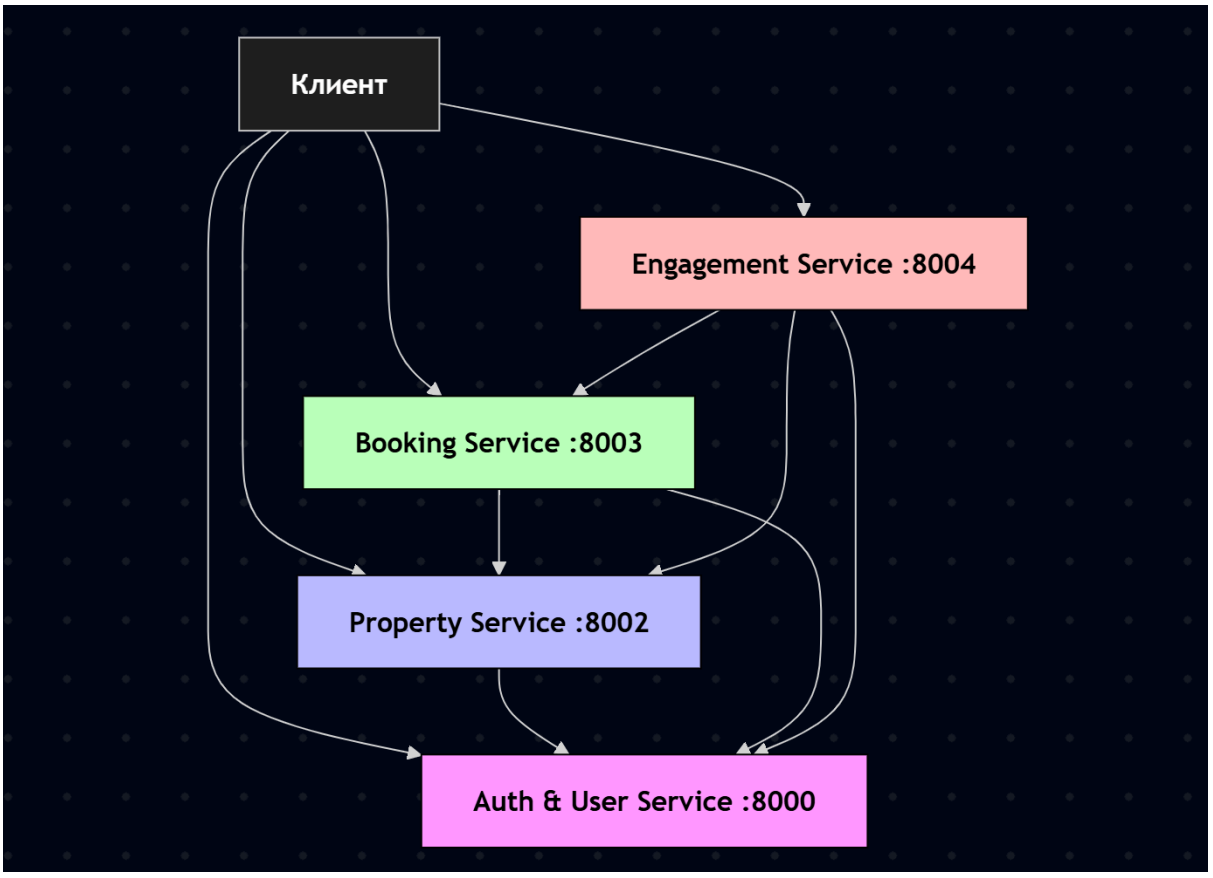
Сервис 4: Engagement Service - объединяет функциональность отзывов, сообщений (чатов) и платежей. Взаимодействует с другими сервисами для валидации существования объектов и прав пользователей.

Каждый сервис имеет собственную базу данных и не имеет прямого доступа к таблицам других сервисов.

2. Взаимосвязь между микросервисами

Взаимодействие построено на синхронных HTTP-вызовах через REST API. Все публичные эндпоинты защищены middleware, которое проверяет JWT через Auth Service.

Схема взаимодействия:



Краткое описание связей:

Все сервисы обращаются к Auth Service для проверки токенов через middleware.

Booking Service вызывает Property Service для проверки существования и получения данных об объекте.

Engagement Service вызывает Property Service для проверки существования объекта при создании отзыва.

Engagement Service вызывает Booking Service для проверки принадлежности бронирования пользователю при создании платежа.

Способы взаимодействия: REST API (HTTP/JSON). Внутренние вызовы используют отдельные internal-эндпоинты, недоступные клиентам напрямую.

3. Разделение базы данных (Database-per-service)

Каждый сервис владеет своей схемой/базой данных и не имеет прямого доступа к чужим таблицам.

- Auth & User Service - база данных auth_user_db:

Таблицы: users

- Property Service - база данных property_db:

Таблицы: properties, property_attributes, property_images, property_locations

- Booking Service - база данных booking_db:

Таблицы: bookings, booking_requests

- Engagement Service -база данных engagement_db:

Таблицы: conversations, messages, payments, reviews

Миграции и управление схемами выполняются отдельно для каждого сервиса.

4. Эндпоинты для межсервисного взаимодействия (internal API)

Для обеспечения взаимодействия между сервисами созданы специальные internal-эндпоинты. Они не документированы в публичной OpenAPI и используются только для внутренних вызовов.

Auth & User Service (порт 8000):

POST /api/internal/verify — проверяет JWT-токен и возвращает данные пользователя (id, email, name, phone, another_contact). Используется middleware всех сервисов.

Property Service (порт 8001):

GET /api/internal/properties/{id} — возвращает базовую информацию об объекте (id, owner_id, title, price_per_month). Используется Booking Service для проверки владельца.

GET /api/internal/properties/{id}/exists — проверяет существование объекта. Используется Engagement Service при создании отзыва.

Booking Service (порт 8002):

GET /api/internal/bookings/{id} - возвращает информацию о бронировании (id, property_id, tenant_id, status). Используется Engagement Service.

POST /api/internal/bookings/verify - проверяет, принадлежит ли бронирование указанному пользователю. Используется PaymentsController.

GET /api/internal/users/{userId}/bookings - возвращает список бронирований пользователя. Используется Engagement Service для получения платежей пользователя.

GET /api/internal/bookings/{id}/exists - проверяет существование бронирования.

Engagement Service (порт 8003):

GET /api/internal/reviews/property/{propertyId} - возвращает список отзывов для объекта. Используется Property Service для отображения отзывов.

GET /api/internal/conversations/property/{propertyId} - возвращает список чатов, связанных с объектом. Используется Property Service.

Варианты ответов и ошибок:

Успешный ответ: HTTP 200 OK с JSON-телом.

Ошибки:

404 Not Found - объект не найден.

401 Unauthorized - невалидный токен (обрабатывается middleware).

403 Forbidden - недостаточно прав.

503 Service Unavailable - сервис-источник недоступен.

5. Авторизация через универсальный middleware

В каждом сервисе (кроме Auth Service) реализован единый middleware, который перехватывает запросы к защищённым эндпоинтам (отмеченным @UseBefore(authMiddleware)). Его алгоритм работы:

Из заголовка Authorization извлекается Bearer-токен.

Middleware отправляет синхронный POST-запрос на внутренний эндпоинт Auth Service /api/internal/verify с телом { "token": "..." }.

Если Auth Service возвращает valid: true, middleware извлекает поле user и сохраняет его в req.user.

Если токен недействителен или сервис недоступен, middleware возвращает клиенту ошибку 401 Unauthorized или 503 Service Unavailable.

Этот подход позволяет централизованно управлять аутентификацией и не дублировать секреты JWT в каждом микросервисе.

Все защищённые эндпоинты в Property, Booking и Engagement Service используют этот middleware без изменений в бизнес-логике — код остался точно таким же, как в монолите.

Вывод

В результате выполнения лабораторной работы монолит разделён на четыре независимых микросервиса. Каждый имеет собственную базу данных, взаимодействует через чётко определённые internal-эндпоинты, а авторизация вынесена в отдельный сервис с универсальным middleware. Архитектура готова к дальнейшему переходу на асинхронное взаимодействие через брокер сообщений (RabbitMQ/Kafka) без изменения бизнес-логики.